

Coding & Solution

Date	06 July 2026
Project Name	Credit Card Approval Prediction System
Prepared By	Joyal Kumar, Keerthi, Yasawini, Nasir, Harika

This section evaluates the quality and completeness of the implemented code and overall solution, based on a direct inspection of the project repository and a live test run of the application (see Performance Testing document for the load-test methodology).

Solution Summary

Field	Details
Repository Link / URL	https://github.com/JoyalKumar1407/Credit-Card-Approval-Prediction
Programming Language(s)	Python
Framework(s) Used	Flask (web app), pandas (data processing), scikit-learn (LogisticRegression, DecisionTreeClassifier, RandomForestClassifier)
Key Features Implemented	- Data cleaning and merging of application_record.csv and credit_record.csv - Good/Bad target label derived from monthly credit STATUS history - Categorical feature encoding via LabelEncoder - Training and comparison of three classifiers (Logistic Regression, Decision Tree, Random Forest) with accuracy, F1 score, and confusion matrix reporting - Best model (Random Forest) persisted to model.pkl - Flask web app with an input form (index.html) and a POST /predict route that returns a Good/Bad Customer prediction (result.html)
Pending / Incomplete Features	- No requirements.txt or setup documentation - LabelEncoder used in training is not persisted/reused at inference, so category encodings could silently drift between runs - No class-imbalance handling (SMOTE/class weighting) despite a likely imbalanced target - "Best model" is hardcoded to Random Forest rather than chosen programmatically from the comparison results - No input validation on the form; server exceptions are shown directly to the user - .venv folder is committed to the repository instead of being git-ignored - Runs only on Flask's development server - not production-ready (see Performance Testing doc)
Setup / Run Instructions	1. Clone the repository: <code>git clone https://github.com/JoyalKumar1407/Credit-Card-Approval-Prediction.git</code> 2. Install dependencies: <code>pip install flask pandas numpy scikit-learn</code> (no requirements.txt is currently provided) 3. (Optional) Retrain the model: <code>python main.py</code> - regenerates model.pkl from the CSV files 4. Run the web app: <code>python app.py</code> 5. Open <code>http://127.0.0.1:5000</code> in a browser and submit the form to get a prediction

Code Quality Checklist

S.No	Criteria	Status
1	Code is modular and organized into functions / classes	No
2	Meaningful variable and function names are used	Yes

S.No	Criteria	Status
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Partial
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments

The application was cloned and run directly to verify functionality: the model loads successfully (with a scikit-learn version-mismatch warning, since model.pkl was trained on 1.5.1 and the test environment used 1.8.0), the home page and prediction form render correctly, and a submitted form returns a valid Good/Bad Customer prediction end-to-end. A real load test (documented separately in the Performance Testing report) showed the app performs well for a single user but degrades under concurrent load, since it currently runs on Flask's development server rather than a production WSGI server. The main gap versus a "modular" codebase is that main.py is a single top-to-bottom script rather than being organized into reusable functions - this is the most impactful next improvement, as it would also let app.py reuse the exact same preprocessing/encoding logic used during training.